Лабораторна робота №8

Формальна специфікація та
верифікація моделі в Alloy

Виконав
Некрасов Артур Дмитрович
Група ІПЗ-23-3

2026

1. Мета і межі моделі

Мета роботи — формально описати структуру замовлення їжі та поведінку диспетчеризації кур'єрів, а потім перевірити властивості безпеки, досяжності й живучості в Alloy. Статична частина продовжує діаграму класів лабораторної роботи №2; динамічна спирається на діаграму станів кур'єра з лабораторної роботи №5 і Z-специфікацію «Диспетчерська» з лабораторної роботи №7.

Перевірка виконана на наданих файлах Лабораторної роботи №8. Моделі відкрито й проаналізовано в Alloy Analyzer 6.2.0 через його CLI із solver SAT4J. Для темпоральних команд використано задані в моделі межі кроків; відсутність контрприкладу означає результат у межах зазначених скінченних областей.

Частина	Що формалізовано	Джерело правил
A. Статика	Клієнт, кур'єр, заклад, страва, замовлення, його позиції та оплата	Діаграма класів Лаб. 2а
Б. Динаміка	Стани кур'єра, пропозиції, призначення замовлення, очікування і доставки	Діаграма станів Лаб. 5; Z-модель Лаб. 7
В. Узгодженість	Пошук прогалин між вимогами, UML, Alloy та Z	Лаб. 2, 4, 5, 6 і 7

2. Частина A. Статична модель

Модель містить вісім сигнатур. Абстрактна Особа має дві неперетинні підсигнатури Клієнт і Кур'єр. Поля використовують множинності one, lone, some та set.

Сигнатура або поле	Множинність	Смисл
Особа; Клієнт, Кур'єр	abstract; extends	Спільний тип і дві ролі, задані як неперетинні підтипи
Заклад.меню	set Страва	Заклад може мати нуль або більше страв
Замовлення.клієнт	one Клієнт	Кожне замовлення має одного клієнта
Замовлення.кур'єр	lone Кур'єр	Замовленню призначено не більше одного кур'єра
Замовлення.позиції	some ПозиціяЗамовлення	У замовленні є щонайменше одна позиція
Замовлення.оплата	one Оплата	Кожне замовлення має одну оплату
ПозиціяЗамовлення.страва	one Страва	Кожна позиція посилається на одну страву

Факт Композиція вимагає, щоб кожна позиція належала рівно одному замовленню. Окремі факти також забороняють спільну оплату і розміщення однієї страви в меню кількох закладів.

```
fact Композиція { all п: ПозиціяЗамовлення | one позиції.п }
fact ОплатаОднаНаЗамовлення { all о: Оплата | one оплата.о }
fact СтраваВМенюОдногоЗакладу { all с: Страва | one меню.с }
fact ОдинЗакладНаЗамовлення { all з: Замовлення | one меню.(з.позиції.страва) }
```

2.1. Результати статичних команд

№	Властивість і команда	Межа	Результат	Висновок
A1	run показатиЗамовлення for 3 expect 1	3	Екземпляр знайдено	Замовлення з двома позиціями й кур'єром допустиме
A2	run постійнийКлієнт for 3 expect 1	3	Екземпляр знайдено	Клієнт може мати два замовлення з кур'єрами
T1	check ПозиціяВОдномуЗамовленні for 5 expect 0	5	Контрприкладу немає	Композиція не дозволяє позицію в двох замовленнях
T2	check БезПовторуСтрави for 5 expect 1	5	Контрприклад знайдено	Дві позиції одного замовлення можуть містити ту саму страву
T3	check ОплатаНеСпільна for 5 expect 0	5	Контрприкладу немає	Факт ОплатаОднаНаЗамовлення забезпечує унікальність оплати

№	Властивість і команда	Межа	Результат	Висновок
C1	run Кур’єрЗамовляєЇжу for 3 expect 0	3	Екземпляра немає	Підтипи Клієнт і Кур’єр неперетинні

Команди run підтвердили здійсненність моделі, тому результати check не є порожньою істиною. У статичному контрприкладі T2 один екземпляр Замовлення має дві різні позиції, але обидві вказують на Страва0.

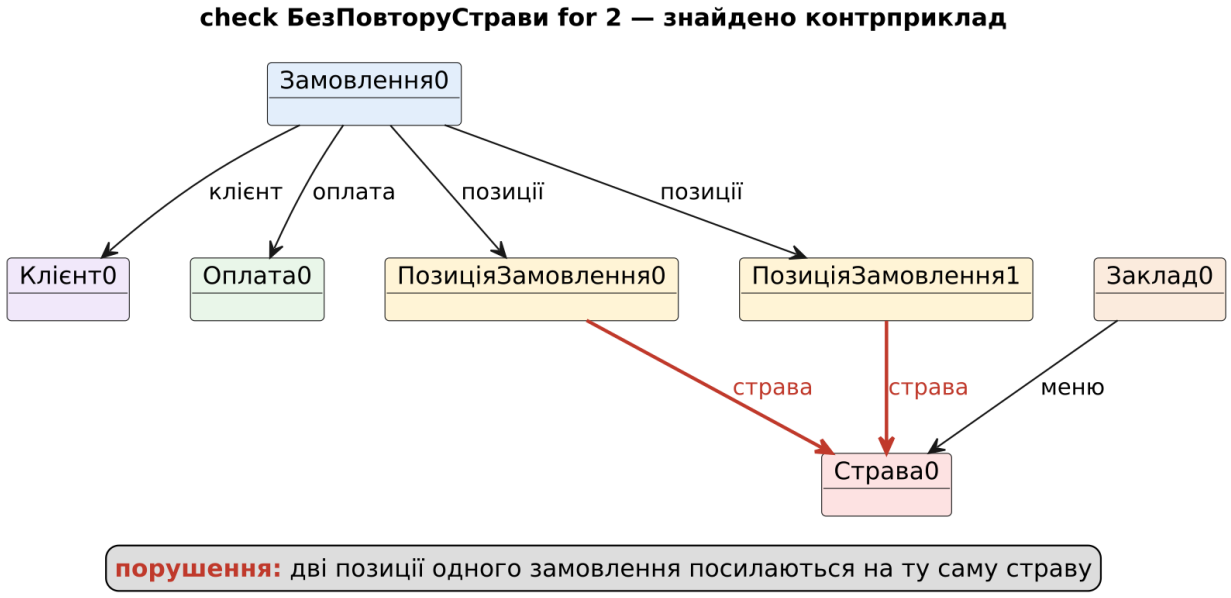


Рисунок 1. Схематичне подання контрприкладу T2, отримане з наданої діаграми PlantUML.

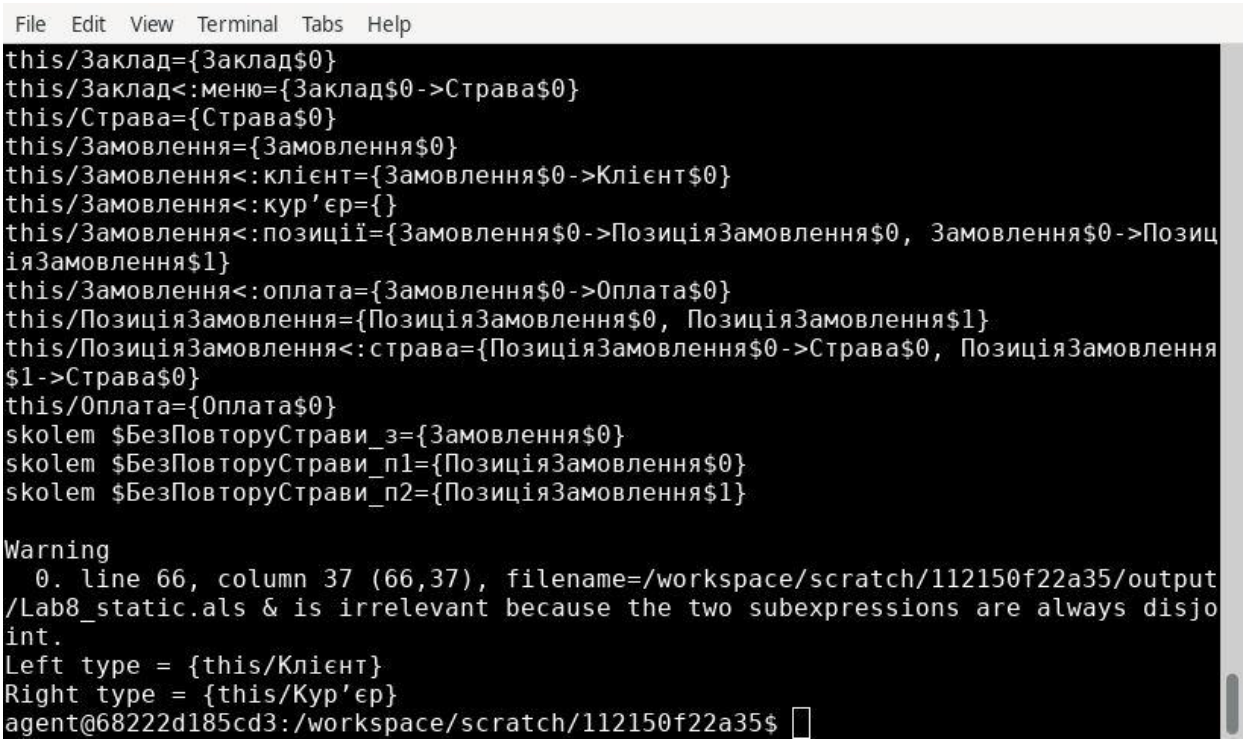


Рисунок 2. Справжній знімок вікна термінала з результатом Alloy CLI для check БезПовторуСтрави.

2.2. Аналіз статичної розбіжності

Контрприклад T2 виявляє прогалину моделі, а не помилку аналізатора: на діаграмі класів Лаб. 2 немає правила унікальності страв у межах замовлення. Якщо одна страва має задаватися кількістю, треба додати інваріант на кшталт self.позиції->isUnique(п | п.страва) та відповідний факт Alloy.

Команда Кур’єрЗамовляєЇжу не має екземпляра, бо підтипи однієї абстрактної сигнатури не перетинаються. Це питання до замовника: чи може та сама людина користуватися сервісом як клієнт і працювати кур’єром? Якщо так, успадкування слід замінити моделлю користувача з ролями.

У цьому звіті знімки екрана підписані за фактичним джерелом. Графічний інтерфейс Alloy не запускається в наявному X11-середовищі, тому наведені скриншоти є справжніми знімками консолі Alloy CLI; контрприклади графічно пояснено окремими діаграмами PlantUML.

3. Частина Б. Динамічна модель

У моделі стану змінними є поля стан і пропозиція кур’єра, поле кур’єра в замовленні та множини ОчікуютьКур’єра і Доставлені. Початковий стан задає кур’єрам статус Офлайн, очищує пропозиції, призначення й обидві множини замовлень. Поле рейтинг є незмінним і має діапазон 10–50; поріг 35 відповідає рейтингу 3,5.

Операції задають передумови й зміни стану; для незмінених полів явно вказані умови рамки. Предикат нічогоНеСталося дозволяє трасі залишатися в тому самому стані.

Операція	Перехід або подія	Передумова та істотна постумова
увійтиНаЗміну	Офлайн → Очікує	Кур’єр виходить на зміну; інші поля збережені
завершитиЗміну	Очікує → Офлайн	Зміна завершується лише зі стану очікування
взятиПерерву	Очікує → Перерва	Перерва можлива без активної доставки чи пропозиції
повернутисяЗПерерви	Перерва → Очікує	Повернення до черги очікування
замовленняОплачено	Зовнішня подія	Оплачене замовлення додається до ОчікуютьКур’єра
надіслатиПропозицію	Очікує → ОтримавПропозицію	Рейтинг ≥ 35; замовлення очікує й ще нікому не запропоноване
відхилив	ОтримавПропозицію → Очікує	Пропозиція знімається, замовлення знову може бути запропоноване
прийняв	ОтримавПропозицію → ПрямуєДоЗакладу	Замовлення призначається кур’єру і виходить із черги
забравЗамовлення	ПрямуєДоЗакладу → ЗабравЗамовлення	Фіксується забір замовлення
вирушив	ЗабравЗамовлення → ПрямуєДоКлієнта	Кур’єр починає доставку клієнтові
доставив	ПрямуєДоКлієнта → Очікує	Замовлення переходить у Доставлені, призначення знімається
нічогоНеСталося	Статус кроку без змін	Усі змінні зберігають попереднє значення

Десять операцій відповідають переходам діаграми станів Лаб. 5. Подія оплати додає замовлення в модель диспетчеризації; вона була відсутня як окрема операція в Z-специфікації Лаб. 7. Операція оновлення GPS не змінює змодельованих даних і поглинається порожнім кроком.

3.1. Інваріант диспетчерської

Інваріант містить сім умов зі схеми стану Z. Перші дві задають ін’єктивність призначення та пропозиції; решта пов’язують поля зі станами й множинами замовлень.

Умова	Зміст
1	Кур’єр має не більше одного призначеного замовлення.
2	Замовлення одночасно запропоновано не більше ніж одному кур’єру.
3	Призначення існує тоді й лише тоді, коли кур’єр у робочому стані.
4	Пропозиція існує тоді й лише тоді, коли кур’єр у стані ОтримавПропозицію.
5	Кожна пропозиція веде до замовлення в ОчікуютьКур’єра.
6	Призначені замовлення не перебувають у черзі очікування.
7	Доставлені замовлення не призначені й не очікують кур’єра.

```
all к: Кур'єр | lone кур'єр.к
all з: Замовлення | lone пропозиція.з
all к: Кур'єр | (some кур'єр.к) iff к.стан in робочі
all к: Кур'єр | (some к.пропозиція) iff к.стан = ОтримавПропозицію
```

Твердження ЗбереженняІнваріанта перевірялося з межею 3 кур'єри, 7-бітовими Int і 12 кроками. Контрприкладу не знайдено. Отже, початковий стан і переходи зберігають інваріант у цій межі; це обмежена автоматична перевірка, а не доведення для необмеженої кількості кур'єрів і довільної довжини траси.

3.2. Перевірки динамічної моделі

Перевірка	Команда та межа	Результат	Тлумачення
B1. Збереження інваріанта	check ЗбереженняІнваріанта; 3 кур'єри, 7 Int, 12 кроків, expect 0	Контрприкладу немає	Переходи зберігають 7 умов інваріанта в заданій межі
B2. Усі стани досяжні	run ДосяжністьУсіхСтанів; 3 кур'єри, 14 кроків, 7 Int, expect 1	Екземпляр знайдений	Усі сім станів кур'єра з'являються в трасі
B3. Повна доставка	run ПовнаДоставка; 2 кур'єри, 10 кроків, 7 Int, expect 1	Екземпляр знайдений	Від оплати можна пройти до Доставлені

Задана ширина Int важлива для рейтингу: за 7 біт Alloy представляє значення від −64 до 63, тому межа 10–50 і поріг 35 не переповнюються. Якщо лишити стандартні 4 біти, перевірялося б інше арифметичне правило.

4. Живучість і припущення справедливості

Властивість B4 формулює вимогу «кожне замовлення, яке очікує кур'єра, колись буде доставлене». Без умов на оточення аналізатор знаходить контрприклад: замовлення оплачене й очікує, але придатного кур'єра немає.

Перевірка	Команда та межа	Результат
B4. Живучість без припущень	check ЗамовленняЗрештоюДоставлене; 2 кур'єри, 7 Int, 10 кроків, expect 1	Контрприклад: замовлення очікує без доступного кур'єра
B5. Живучість за C1–C3	check ЖивучістьЗаСправедливості; 2 кур'єри, 7 Int, 10 кроків, expect 1	Контрприклад: нескінченний цикл відхилень
B6. Живучість з ОбмеженеВідхилення	check ЖивучістьЗПравилом; 2 кур'єри, 7 Int, 10 кроків, expect 0	Контрприкладу немає в заданій межі

Позначення	Припущення	Чому воно потрібне
C1	Існує кур'єр з рейтингом не нижче 35, який нескінченно часто перебуває в стані Очікує.	Модель не повинна вимагати доставки без доступного персоналу.
C2	Якщо замовлення і придатний кур'єр нескінченно часто доступні, диспетчер нескінченно часто надсилає пропозицію.	Виключається бездіяльність диспетчера.
C3	Кур'єр, що розпочав робочу частину маршруту, зрештою повертається до стану Очікує.	Задається завершення доставки для кур'єра, який її прийняв.

B5 перевіряє ту саму вимогу за C1–C3. Контрприклад — нескінченна траса-ласо: кур'єр очікує, отримує пропозицію, відхиляє її й повторює цей цикл. Припущення справедливості виконуються, але конкретне замовлення не потрапляє до Доставлені.

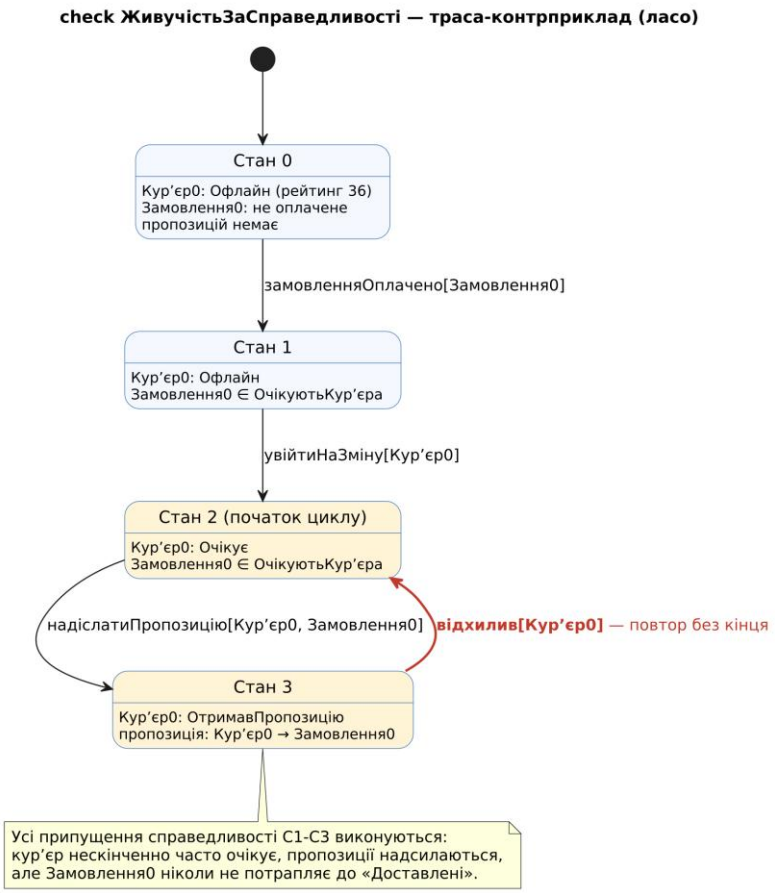


Рисунок 3. Діаграма траси-ласо за наданим файлом PlantUML: повторне відхилення пропозиції.

```
File  Edit  View  Terminal  Tabs  Help
seq/Int={0, 1}
String={}
none={}
this/Офлайн={Офлайн$0}
this/Очікує={Очікує$0}
this/ОтримавПропозицію={ОтримавПропозицію$0}
this/ПрямуєДоЗакладу={ПрямуєДоЗакладу$0}
this/ЗабравЗамовлення={ЗабравЗамовлення$0}
this/ПрямуєДоКлієнта={ПрямуєДоКлієнта$0}
this/Перерва={Перерва$0}
this/СтанКур'єра={Офлайн$0, Очікує$0, ОтримавПропозицію$0, ПрямуєДоЗакладу$0, За
бравЗамовлення$0, ПрямуєДоКлієнта$0, Перерва$0}
this/Кур'єр={Кур'єр$0}
this/Кур'єр<:рейтинг={Кур'єр$0->36}
this/Кур'єр<:стан={Кур'єр$0->ОтримавПропозицію$0}
this/Кур'єр<:пропозиція={Кур'єр$0->Замовлення$0}
this/Замовлення={Замовлення$0}
this/Замовлення<:кур'єр={}
this/ОчікуютьКур'єра={Замовлення$0}
this/Доставлені={}
skolem $Справедливість_к={Кур'єр$0}
skolem $ЖивучістьЗаСправедливості_з={Замовлення$0}

agent@68222d185cd3:/workspace/scratch/112150f22a35$
```

Рисунок 4. Справжній знімок вікна термінала з контрприкладом Alloy CLI для перевірки живучості за C1–C3.

4.1. Бізнес-правило для живучості

Після додавання ОбмеженеВідхилення — пропозиція не може з'являтися без кінця без призначення замовлення — перевірка B6 у межі 2 замовлення, 7 Int і 10 кроків не знайшла контрприкладу. Це робоча вимога до процесу призначення, а не універсальне доведення. Практичний механізм може

рахувати відмови, підвищувати оплату або передавати замовлення оператору; точний варіант потрібно погодити із замовником і відобразити в UML та Use Case.

5. Мутаційний експеримент

Для експерименту у копії динамічної моделі прибрано передумову no пропозиція.з в операції надіслатиПропозицію. Вона забороняла повторно запропонувати замовлення, яке вже має активну пропозицію. Перевірку ЗбереженняІнваріанта повторено з межею 3 кур'єри, 7 Int і 12 кроками; Alloy знайшов контрприклад.

Крок	Подія	Стан після події
0	Початковий стан	Два кур'єри офлайн; замовлення ще не оплачено.
1	замовленняОплачено	Замовлення0 входить до ОчікуютьКур'єра.
2	увійтиНаЗміну[Кур'єр0]	Кур'єр0 переходить до Очікує.
3	надіслатиПропозицію[Кур'єр0, Замовлення0]	Кур'єр0 отримав пропозицію.
4	увійтиНаЗміну[Кур'єр1]	Кур'єр1 також переходить до Очікує.
5	надіслатиПропозицію[Кур'єр1, Замовлення0]	Обидва кур'єри мають пропозицію на те саме замовлення; порушено ін'єктивність.

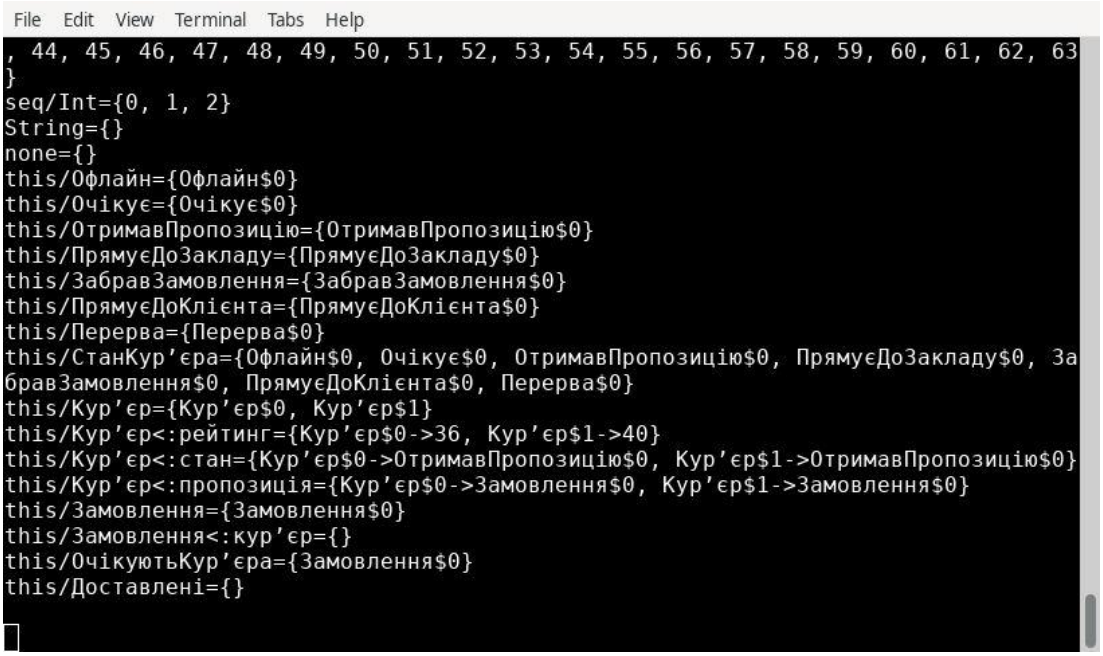


Рисунок 5. Справжній знімок Alloy CLI: після мутації те саме замовлення запропоновано двом кур'єрам.

Експеримент показує, що перевірка безпеки чутлива до втрати передумови: після її вилучення порушення з'являється на п'ятому переході. Це конкретизує неоднозначність Н3 з Лаб. 7 щодо паралельних пропозицій.

6. Узгодженість моделей і розбіжності

Результати Alloy зіставлено з UML-моделями лабораторних робіт №2, 4, 5, 6 та Z-специфікацією лабораторної роботи №7. Для кожного пункту позначено тип питання й потрібну дію.

№	Розбіжність	Тип	Пропозиція
P1	Діаграма класів допускає дві позиції з тією самою стравою в одному замовленні (T2).	Дефект моделі Лаб. 2	Додати інваріант унікальності страв у позиціях замовлення.
P2	Підтипи Клієнт і Кур'єр не перетинаються; кур'єр не може бути клієнтом.	Питання до замовника	У разі потреби перейти до сутності Користувач із ролями.
P3	Цикл пропозицій може тривати без обмеження, тому живучість не гарантована (B5).	Прогалина вимог Лаб. 4	Визначити ліміт відмов або альтернативну ескалацію.
P4	Окремий boolean статусДоступності дублює стан кур'єра й може йому суперечити.	Дефект моделі Лаб. 2	Видалити атрибут або зробити його похідним від стану.
P5	Немає переходу тайм-ауту, якщо кур'єр не відповідає на пропозицію.	Дефект моделі Лаб. 5	Додати перехід за таймером і описати поведінку диспетчера.
P6	Подія оплати є в Alloy, але відсутня окремою операцією в Z.	Свідоме спрощення Лаб. 7	Додати операцію ЗамовленняОплачено до схеми диспетчеризації.
P7	У діаграмі послідовності Лаб. 6 є клас Пропозиція, якого немає в діаграмі класів.	Спрощення або дефект Лаб. 2	Уточнити: окремий клас чи зв'язок Кур'єр—Замовлення.

7. Опис обраного фрагмента та його зв’язку з Лаб. 2, Лаб. 5 і Лаб. 7

Обраний фрагмент системи — диспетчеризація замовлення їжі: після оплати замовлення потрапляє до черги, кур’єрові надсилається пропозиція, а прийняте замовлення проходить шлях від закладу до клієнта. У роботі розглянуто і структуру замовлення, і послідовність зміни стану кур’єра.

Простежуваний зв’язок із попередніми лабораторними

Джерело	Що взято з попередньої роботи	Як використано й перевірено в Лаб. 8
Лаб. 2 — діаграма класів	Предметні сутності Клієнт, Кур’єр, Заклад, Страва, Замовлення, ПозиціяЗамовлення й Оплата; зв’язки між ними та їхні кратності.	Структуру перенесено в сигнатури, поля й факти Alloy. Перевірено композицію позицій та унікальність оплати. Контрприклад T2 показав, що діаграма не забороняє двом позиціям замовлення посилатися на ту саму страву.
Лаб. 5 — діаграма станів	Життєвий цикл кур’єра: Офлайн, Очікує, ОтримавПропозицію, ПрямуєДоЗакладу, ЗабравЗамовлення, ПрямуєДоКлієнта й Перерва; події та переходи між станами.	Стани подано змінним полем, а переходи — операціями Alloy (вихід на зміну, перерва, пропозиція, прийняття, відмова, забір і доставка). Перевірка B2 знайшла трасу з усіма сімома станами, B3 — повний шлях до доставки.
Лаб. 7 — Z-специфікація «Диспетчерська»	Стан диспетчеризації, черга замовлень, пропозиції та призначення кур’єрам; обмеження на одночасні призначення й пропозиції.	Стан, початкову умову, переходи й сім умов інваріанта закодовано в динамічній моделі Alloy. Перевірка B1 тестує збереження інваріанта; B4–B6 аналізують живучість. Мутаційний експеримент підтверджує, що вилучення заборони повторної пропозиції допускає подвійне призначення.

Отже, Лаб. 8 поєднує три рівні попередньої роботи: структуру даних із Лаб. 2, поведінкові переходи з Лаб. 5 і формальні правила диспетчеризації з Лаб. 7. Alloy дає змогу перевірити їх у спільній моделі та виявити прогалини, які окремі UML-діаграми чи Z-опис не виявили.

Висновки

Alloy знайшов конкретну структурну прогалину: дві позиції одного замовлення можуть посилатися на одну страву. У динамічній частині досяжність семи станів і повний шлях доставки підтверджено прикладами; інваріант збережено в межах перевірки. Властивість живучості не виконується без припущень і залишається хибною навіть за справедливості, якщо кур’єр нескінченно відхиляє пропозицію. Правило обмеження повторних відмов усуває знайдений контрприклад у заданій межі. Мутаційний запуск продемонстрував, що вилучення захисної передумови відкриває шлях до подвійної пропозиції.

Автоматична перевірка доповнила ручну звірку з UML та Z: вона дала мінімальні приклади дозволених станів і точні траси порушення, а також виявила місця, де вимоги потребують уточнення. Усі позитивні результати є обмеженими областю атомів і кількістю кроків, указаними в командах.